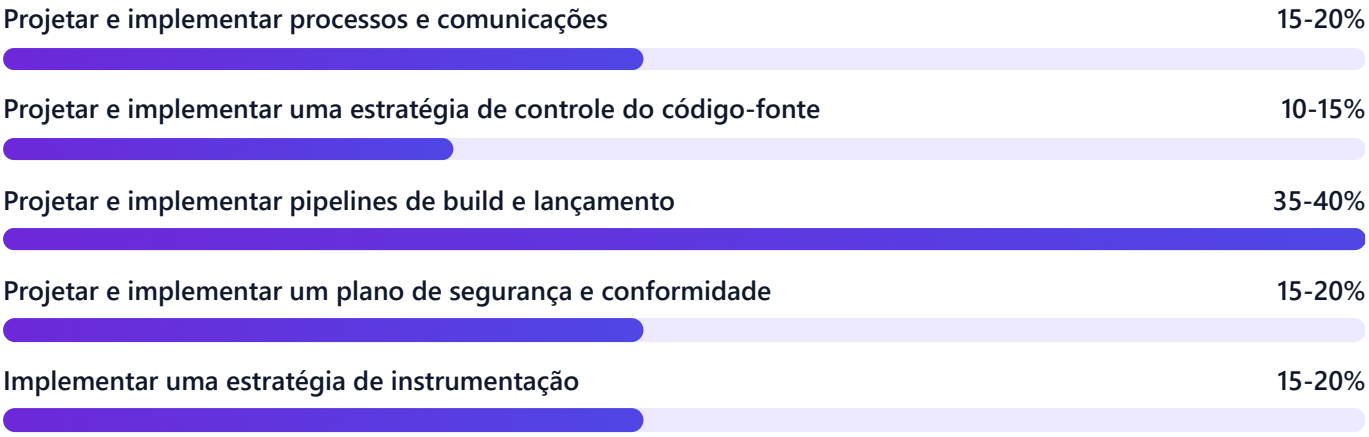
Strecht Simulados

Resumo de bolso AZ-401

Domínios cobrados, as dicas de prova mais importantes e 10 questões comentadas para treinar.

300	6	70%
questões no curso	provas completas	nota de corte

Como é a prova AZ-401



Material independente de preparação, sem afiliação ou endosso de Microsoft. As questões são inéditas e não reproduzem a prova real.

Pronto para o exame AZ-401?

Treine com as 300 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →

Dicas de prova por domínio

Projetar e implementar processos e comunicações

- Hooks do cloud agent: JSON na branch padrão, bash e powershell, timeout padrão de 30 s.
- Instruções da organização: Copilot Chat, code review e cloud agent no GitHub.com.
- Cliente MCP próprio: escopos OAuth mínimos; escopo não supera a permissão do usuário.
- Requisito verificado por caso de teste: vínculo Tested by.
- Mudança formal, risco, revisão e auditoria apontam para CMMI; User Story para Agile; PBI para Scrum; Issue para Basic.
- Licença do repositório: arquivo LICENSE na raiz.
- Visão de calendário de várias equipes com dependências: Delivery Plans.
- Cloud agent + MCP do Azure DevOps: azure/login com OIDC no copilot-setup-steps e servidor ado.

Projetar e implementar pipelines de build e lançamento

- Ambiente do Copilot code review separado do agente: copilot-code-review.yml.
- CI só para uma pasta: trigger com branches e paths include.
- Fila ordenada sem perder execuções: queue: max, sem cancel-in-progress.
- Restringir runners a alguns repositórios: runner group.
- Repositório público: evite runner auto-hospedado; prefira efêmeros.
- Registro público instável: upstream sources + um único feed no arquivo de configuração.
- App Service sem downtime e com rollback rápido: slot de staging + swap.
- Versão por data no Azure Pipelines: $(Date:yyyyMMdd).$(Rev:r), o padrão do run number.

Projetar e implementar uma estratégia de controle do código-fonte

- Segredo no histórico: revogue antes de pensar em reescrever.
- Arquivo grande no histórico: congelar, reescrever e push forçado (Force push).
- PRs testados contra main antiga quebrando a main: exigir branch atualizada antes do merge.
- Exceção controlada a um ruleset: lista de bypass com equipes ou apps específicas.
- Revisão automática do Copilot em todos os PRs: ruleset com Automatically request Copilot code review.
- Várias tarefas ou agentes em paralelo no mesmo repositório: git worktree.
- Integração frequente na main + feature flags + release branch cortada da main = trunk-based.
- Branch excluída no Azure Repos: restaurar pela pesquisa de branches excluídas; não há retenção.

Projetar e implementar um plano de segurança e conformidade

- OIDC para job de pull_request sem environment: subject repo:ORG/REPO:pull_request.
- Conector Azure DevOps no Defender for Cloud: PCA + Basic (Stakeholder não serve) + OAuth ligado + Contributor.
- Varredura só de IaC para o Defender for Cloud: MSDO com a categoria IaC.
- Bypass de push protection: o push passa, mas fica um alerta para revisão.
- Sugestão pontual: Copilot Autofix. Correção completa com PR: agentic autofix.
- OIDC com environment: subject repo:ORG/REPO:environment:NOME.
- Bloquear merge por alertas do CodeQL: ruleset Require code scanning results.
- Default setup do CodeQL com rede privada: pool auto-hospedado e tarefas do Marketplace liberadas.

Implementar uma estratégia de instrumentação

- Custom instructions do Observability Agent: orientam correlação e issues; não dão permissões.
- Onde o trabalho está parando no quadro: Cumulative Flow Diagram.
- Investigar um alerta com o Observability Agent: Investigate no alerta e salvar como issue.
- Por que o agente mudou isso: session logs, acessíveis pelo link no commit.
- SRE Agent: Review em produção, Autonomous em não produção, configurado por response plan.
- Evolução de itens abertos e concluídos no GitHub Projects: gráfico histórico (Burn up).
- Validar uma correção durante a liberação, ao vivo: Live metrics.
- Stream do audit log quebrado: aviso diário e até seis dias para corrigir.

10 questões para treinar

Responda antes de olhar o gabarito, no fim do material.

Questão 1 · Projetar e implementar pipelines de build e lançamento

O environment de produção da Telefonia Fala Mais tem a verificação Exclusive lock. Várias execuções do pipeline chegam ao stage de produção quase ao mesmo tempo, e a equipe quer que todas sejam implantadas, uma de cada vez, na ordem em que chegaram. O que configurar no stage?

A. condition: always() no stage
B. lockBehavior: runLatest
C. lockBehavior: sequential
D. dependsOn: [] no stage de produção

Questão 2 · Projetar e implementar pipelines de build e lançamento

A Rede Hospitalar Vida Longa usa pools de agentes com conjuntos de dimensionamento de VMs (VMSS) na própria assinatura do Azure e gasta muito tempo mantendo imagens e escala. Ela avalia o Managed DevOps Pools. Qual diferença de arquitetura é correta?

A. No Managed DevOps Pools, as VMs dos agentes ficam em uma assinatura da Microsoft, não na sua
B. O Managed DevOps Pools substitui os agentes por runners do GitHub Actions hospedados no Azure
C. No Managed DevOps Pools, as VMs ficam na sua assinatura, mas com escala manual
D. O Managed DevOps Pools só oferece as imagens padrão dos agentes hospedados, sem personalização

Questão 3 · Projetar e implementar pipelines de build e lançamento

A equipe de segurança da Fábrica de Software Prisma Dev recompilou internamente uma versão corrigida de um pacote open source e a publicou em um feed interno. O feed principal dos projetos tem upstreams para esse feed interno e para o registro público. Como garantir que a versão corrigida seja a usada?

A. Colocar o registro público antes do feed interno, pois públicos vêm primeiro
B. Adicionar os dois feeds no arquivo de configuração de cada projeto
C. Publicar a versão corrigida com número menor que o da versão pública
D. Colocar o feed interno antes do registro público na ordem dos upstreams

Questão 4 · Projetar e implementar pipelines de build e lançamento

A Siderúrgica Ferro Novo usa GitHub Enterprise Cloud. Os workflows de build precisam de mais memória e CPU, e o firewall de um parceiro só aceita conexões de endereços IP conhecidos. A empresa não quer administrar máquinas próprias para os runners. Qual opção atende?

A. Runners auto-hospedados em VMs da empresa com IP público fixo
B. Runners padrão hospedados pelo GitHub com cache de dependências
C. Os larger runners hospedados pelo GitHub, com IP estático
D. Actions Runner Controller em um cluster Kubernetes da empresa

Questão 5 · Projetar e implementar pipelines de build e lançamento

A equipe da Biblioteca de Componentes Aurora publica um pacote npm com escopo da organização no GitHub Packages sempre que uma release é publicada. O workflow atual falha com erro de autenticação no npm publish, e a segurança proíbe PATs. A equipe revisou o exemplo da documentação e quer saber quais configurações do job são necessárias para publicar com o token do próprio workflow. Qual combinação está correta?

A. packages: write nas permissões, setup-node com registry-url do GitHub Packages e NODE_AUTH_TOKEN com o GITHUB_TOKEN
B. contents: write nas permissões, setup-node sem registry-url e um PAT clássico em NODE_AUTH_TOKEN para o npm
C. packages: read nas permissões, publicação pelo npm login interativo no runner e o GITHUB_TOKEN no package.json
D. id-token: write nas permissões e publicação por OIDC direto no npm, sem setup-node nem NODE_AUTH_TOKEN

Questão 6 · Projetar e implementar um plano de segurança e conformidade

Uma Azure Function da Transportes Expresso Sul precisa gravar em uma conta de armazenamento. O time de segurança proíbe segredos e chaves em configurações do aplicativo e quer que a rotação de credenciais seja responsabilidade do Azure. Qual identidade usar?

A. Identidade gerenciada da Function com papel RBAC no armazenamento
B. Chave de acesso da conta de armazenamento referenciada a partir do Key Vault
C. Entidade de serviço com certificado carregado e renovado pela equipe de operações
D. Entidade de serviço com segredo de cliente guardado nas configurações do app

Questão 7 · Projetar e implementar pipelines de build e lançamento

O feed do Azure Artifacts da Indústria de Autopeças Engrenagem está crescendo rápido, e o dono do feed habilitou retention policies com máximo de 20 versões por pacote e 30 dias para pacotes baixados recentemente. A equipe teme perder versões promovidas para a view @Release e versões ainda usadas. Como a política se comporta?

A. Exclui definitivamente as versões no mesmo instante, sem passar pela Lixeira do feed
B. Exclui as versões mais antigas assim que passam de 20, incluindo as que estão na view @Release
C. Exclui uma versão só quando o máximo é atingido e ela não foi baixada no período; versões em views ficam isentas
D. Exclui todas as versões não baixadas em 30 dias, mesmo que o pacote tenha menos de 20 versões

Questão 8 · Projetar e implementar pipelines de build e lançamento

O pipeline multiestágio da Companhia de Logística Rota Norte tem um stage Testes. Depois dele, os stages DeployEUA e DeployBrasil devem rodar em paralelo, e um stage Validacao deve rodar só quando os dois deploys terminarem. Como declarar as dependências?

A. DeployBrasil com dependsOn: DeployEUA e Validacao com dependsOn: DeployBrasil
B. Os dois deploys com dependsOn: Testes e a Validacao com dependsOn listando ambos
C. Todos os stages com dependsOn: [] para rodarem juntos e Validacao com condition: always()
D. Validacao com dependsOn: Testes, e os deploys sem dependsOn para começarem primeiro

Questão 9 · Projetar e implementar pipelines de build e lançamento

A API de cotações da Corretora Bolsa Viva roda em 12 VMs registradas em um environment do Azure Pipelines. A equipe quer atualizar no máximo três VMs por vez, para manter capacidade de atendimento durante a implantação. Qual estratégia de deployment job usar?

A. strategy: rolling com maxParallel: 3 no deployment job
B. strategy: canary com increments de 100 no environment
C. strategy: runOnce com deploy em todas as VMs de uma vez
D. strategy: matrix com uma combinação por VM do environment

Questão 10 · Implementar uma estratégia de instrumentação

A aplicação de pedidos do Atacarejo Bom Preço tem sete microsserviços instrumentados com Application Insights. Os usuários relatam erros intermitentes e o time quer ver, em uma única visão, os componentes, as chamadas entre eles e onde estão as falhas e a lentidão. Qual recurso usar?

A. Testes de disponibilidade com alertas por região de origem
B. Metrics explorer com a CPU de cada nó do cluster de contêineres
C. Live metrics do Application Insights com filtros por servidor
D. Application map do Application Insights, com os nós e chamadas

Gabarito comentado

1. C) lockBehavior: sequential
Exclusive lock com todas as execuções em fila: lockBehavior sequential.

2. A) No Managed DevOps Pools, as VMs dos agentes ficam em uma assinatura da Microsoft, não na sua
Managed DevOps Pools: evolução do VMSS, infraestrutura em assinatura da Microsoft.

3. D) Colocar o feed interno antes do registro público na ordem dos upstreams
Pacote open source corrigido internamente: upstream interno antes do público.

4. C) Os larger runners hospedados pelo GitHub, com IP estático
Mais hardware e IP estático sem gerenciar máquinas: larger runners.

5. A) packages: write nas permissões, setup-node com registry-url do GitHub Packages e NODE_AUTH_TOKEN com o GITHUB_TOKEN
npm no GitHub Packages: packages: write + registry-url + NODE_AUTH_TOKEN = GITHUB_TOKEN.

6. A) Identidade gerenciada da Function com papel RBAC no armazenamento
Código em recurso do Azure acessando outro serviço do Azure: identidade gerenciada.

7. C) Exclui uma versão só quando o máximo é atingido e ela não foi baixada no período; versões em views ficam isentas
Retenção no Azure Artifacts: limite de versões E sem download recente; views isentas.

8. B) Os dois deploys com dependsOn: Testes e a Validacao com dependsOn listando ambos
Paralelo depois junta: fan-out e fan-in com dependsOn.

9. A) strategy: rolling com maxParallel: 3 no deployment job
Atualizar VMs em lotes: strategy rolling com maxParallel.

10. D) Application map do Application Insights, com os nós e chamadas
Ver topologia, dependências e pontos de falha entre microsserviços: Application map.

Pronto para o exame AZ-401?

Treine com as 300 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →